07

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 1

CPU 바운드와 워커 스레드

제1부 기반: 백엔드와 Node.js 런타임 · 2장 이벤트 루프와 비동기 I/O

무거운 계산을 이벤트 루프 밖으로 보내는 방법과 그 비용



© 한상훈 NestJS 백엔드 마스터

로스팅을 시작한 직원

→ 직원 한 명, 샷 걸어 두고 주문 받기 → 잘 돌아감

→ 원두 로스팅 시작 → 20분 내내 로스터 앞

→ 그동안 주문, 음료 전달 전부 정지



© 한상훈 NestJS 백엔드 마스터

뒷방 직원

→ 로스팅은 뒷방 직원에게

→ 카운터 직원은 계속 주문

→ 뒷방 직원이 끝나면 원두만 전달

→ 서버의 로스팅: 엑셀 내려받기, PDF 티켓 생성, 단체 예매 CSV 검사



© 한상훈 NestJS 백엔드 마스터

정산 리포트

report.mjs

```
// 예매 건을 구역별로 모아 합계와 상위 금액을 냄
export function buildReport(count) {
  const zones = new Map();
  for (let i = 0; i < count; i++) {
    const zone = 'ABCDEFGH'[Math.floor(rand() * 8)];
    const price = 50000 + Math.floor(rand() * 15) * 10000;
    const z = zones.get(zone) ?? { zone, sold: 0, amount: 0, prices: [] };
    z.sold++; z.amount += price;
    if (i % 10 === 0) z.prices.push(price);
    zones.set(zone, z);
  }
  return [...zones.values()].map(z => ({
    zone: z.zone, sold: z.sold, amount: z.amount,
    top: z.prices.sort((a, b) => b - a).slice(0, 3),
  }));
}
```

→ rand: 고정 시드 난수 (전체 코드는 부록)

© 한상훈 NestJS 백엔드 마스터

2천만 건 집계 1,375ms

터미널

```
buildReport 2천만 건 1375ms
```

1,375ms

→ DB 조회 없음, 순수 계산

데이터 블록: 2026-10-26 강사 PC, Node v24.11.0 (부록 시트), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

리포트가 도는 동안의 /health

터미널

```
$ node probe.mjs /report
/report 응답: 1585ms
그동안 /health 다섯 번: 1439, 2, 2, 3, 2ms
```

→ /report 호출 0.1초 뒤부터 0.2초 간격으로 /health 다섯 번

→ 첫 번째: 리포트가 끝날 때까지 대기

→ 평소 초당 1,000 요청이면: 1.4초 동안 1,400개가 쌓임

→ 찾는 법: 이벤트 루프 지연 최댓값이 튄 시각의 요청 로그

© 한상훈 NestJS 백엔드 마스터

CPU 바운드와 I/O 바운드

구분	시간을 쓰는 곳	예
CPU 바운드	계산	집계, 정렬, 해시, 압축, 이미지 변환
I/O 바운드	기다림	DB 쿼리, 외부 API, 파일 읽기

→ CPU가 빨라지면 빨라지는 작업 / 상대가 빨라져야 빨라지는 작업

© 한상훈 NestJS 백엔드 마스터

걸린 시간과 CPU 시간

```
const c0 = process.cpuUsage(); const t0 = performance.now();
await fn();
const c = process.cpuUsage(c0);
// c.user + c.system: 마이크로초
```

○ ○ ○ 터미널

```
DB 응답 200ms 기다리기: 걸린 시간 214ms, CPU 사용 0ms
비밀번호 해시 (pbkdf2Sync 200만 회): 걸린 시간 203ms, CPU 사용 203ms
```

→ CPU 시간 ≈ 걸린 시간 → CPU 바운드

→ CPU 시간 ≈ 0 → I/O 바운드

© 한상훈 NestJS 백엔드 마스터

async를 붙여도

→ `async function buildReportAsync() { ... 같은 반복문 ... }`

→ await가 없으면 처음부터 끝까지 한 번에 실행

→ async: 결과를 프로미스로 감쌀 뿐, 계산을 다른 곳으로 보내지 않음

→ I/O 바운드에 워커 추가: 기다림은 그대로, 비용만 증가

→ 섞인 작업: DB에서 읽기(I/O) + 집계(CPU) → 나눠서 측정, 계산만 워커로, 또는 DB가 집계

© 한상훈 NestJS 백엔드 마스터

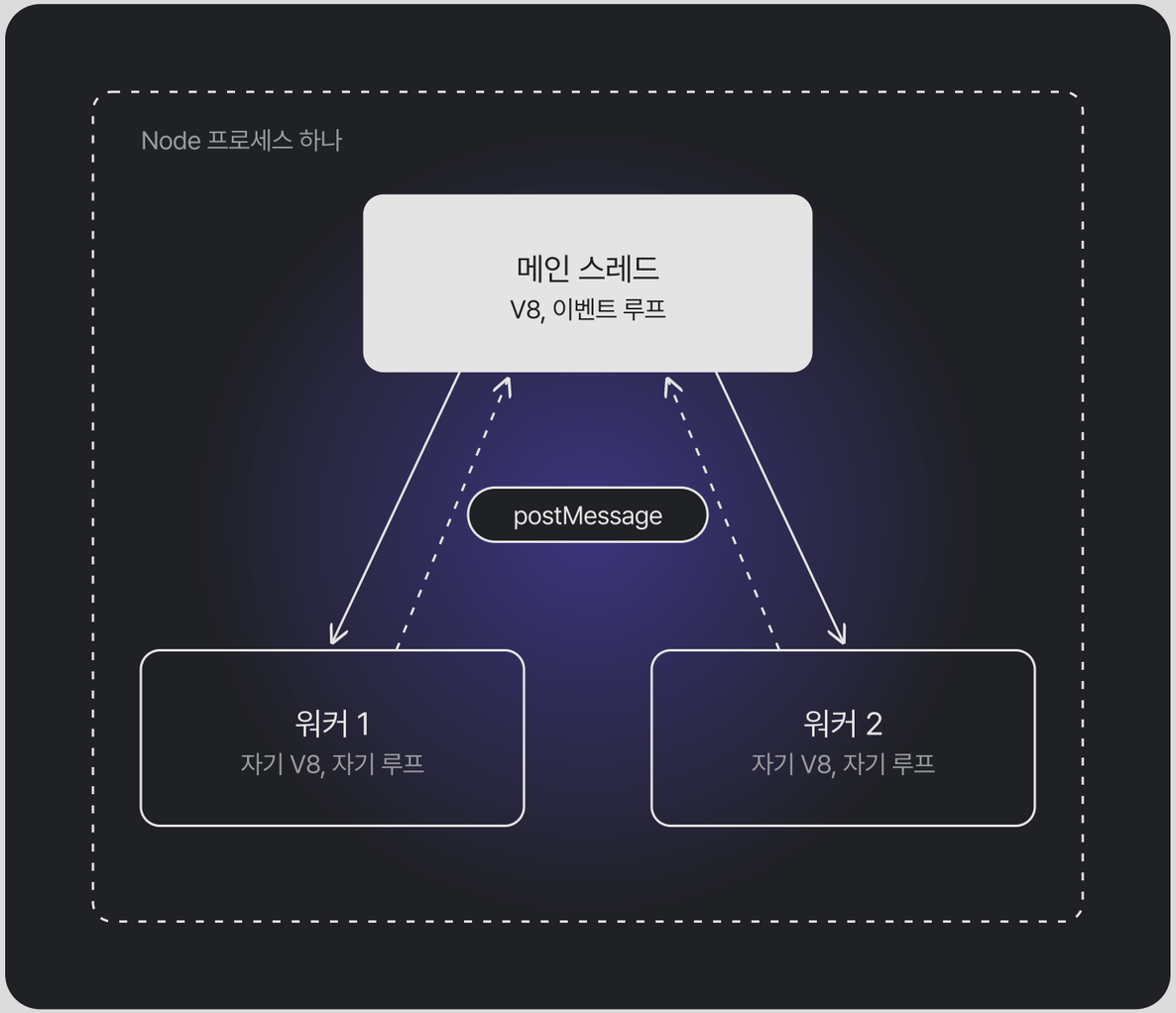
워커 스레드

→ `node:worker_threads` 모듈

→ 같은 프로세스 안의 별도 스레드

→ 워커마다: 자기 V8 인스턴스, 자기 이벤트 루프

→ 메인 스레드와 대화: `postMessage`로 메시지 주고받기



© 한상훈 NestJS 백엔드 마스터

리포트를 워커로

```
// report-worker.mjs
import { parentPort } from 'node:worker_threads';
import { buildReport } from './report.mjs';
parentPort.on('message', (count) => parentPort.postMessage(buildReport(count)));
```

```
// server.mjs
function inWorker(count) {
  return new Promise((resolve, reject) => {
    const w = new Worker(new URL('./report-worker.mjs', import.meta.url));
    w.once('message', (r) => { resolve(r); w.terminate(); });
    w.once('error', reject);
    w.postMessage(count);
  });
}
```

© 한상훈 NestJS 백엔드 마스터

워커가 계산하는 동안의 /health

터미널

```
$ node probe.mjs /report-worker
그동안 /health 다섯 번: 12, 6, 2, 1, 1ms
/report-worker 응답: 1282ms
```

→ 리포트 시간은 그대로, 메인 스레드는 계속 응답

→ 워커 안의 에러: 메인에 error 이벤트, 메인 스레드는 계속 동작

© 한상훈 NestJS 백엔드 마스터

스레드 풀과 워커 스레드

구분	스레드 풀	워커 스레드
만드는 쪽	libuv (기본 4개)	우리 코드
실행하는 것	정해진 일 (파일, DNS, 해시, 압축)	우리가 쓴 자바스크립트
대화	끝나면 콜백	`postMessage`

→ NestJS: 워커를 다루는 서비스를 주입, 앱 종료 훅에서 워커 종료

© 한상훈 NestJS 백엔드 마스터

워커 하나의 값

터미널

```
$ node cost.mjs
워커 생성부터 첫 답까지: 중앙값 20.4ms (10회, 최소 16.9, 최대 30.8)
```

→ 새 V8 인스턴스 + 새 이벤트 루프 + 모듈 다시 불러오기

20.4ms

데이터 블록: 2026-10-26 강사 PC, Node v24.11.0 (부록 시트), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

워커 수와 메모리

띄워 둔 워커	프로세스 메모리(RSS)
0개	66MB
10개	174MB
50개	625MB

→ 워커 하나당 약 11MB (아무 일도 안 하는 워커)

→ 리포트 요청 50개가 동시에 오면 → 워커 50개

© 한상훈 NestJS 백엔드 마스터

메시지는 복사

→ `postMessage`: 값을 복사해서 전달 (structured clone)

→ 객체, 배열, Map: 복사

→ `ArrayBuffer`: transfer 목록에 넣으면 복사 없이 넘김, 보낸 쪽은 빈 버퍼

→ 함수, 클래스 인스턴스의 메서드: 못 넘김

→ 큰 데이터는 넘기지 말고 조건만 전달 (예: 공연 ID, 기간) → 워커가 직접 조회

© 한상훈 NestJS 백엔드 마스터

100MB 복사와 이전

```
w.postMessage(buf);              // 복사
w.postMessage(buf, [buf]);       // 이전(transfer)
```

○ ○ ○ 터미널

```
100MB 복사: 28.7ms, 보낸 뒤 메인 쪽 byteLength 104857600
100MB 이전(transfer): 1.2ms, 보낸 뒤 메인 쪽 byteLength 0
```

→ 복사하는 28.7ms 동안 메인 스레드도 멈춤

© 한상훈 NestJS 백엔드 마스터

워커 풀

```
export class WorkerPool {
  #idle = [];
  #waiting = [];   // 빈 워커를 기다리는 작업
  constructor(file, size) {
    for (let i = 0; i < size; i++) this.#idle.push(new Worker(file));
  }
  run(data) {
    return new Promise((resolve, reject) => {
      this.#waiting.push({ data, resolve, reject });
      this.#next();
    });
  }
  #next() {
    if (!this.#idle.length || !this.#waiting.length) return;
    const w = this.#idle.pop();
    const job = this.#waiting.shift();
    w.once('message', (r) => { this.#idle.push(w); job.resolve(r); this.#next(); });
    w.postMessage(job.data);
  }
}
```

→ 에러 처리 생략 (부록)

→ 워커가 죽으면: 그 자리에 새 워커, 그 작업은 실패 처리, 작업마다 시간 제한

© 한상훈 NestJS 백엔드 마스터

풀 4개에 리포트 8개

터미널

```
$ node pool.mjs
이 PC의 availableParallelism(): 16
리포트 #2 끝: 789ms
리포트 #3 끝: 835ms
리포트 #4 끝: 835ms
리포트 #1 끝: 901ms
리포트 #5 끝: 1630ms
리포트 #7 끝: 1638ms
리포트 #6 끝: 1648ms
리포트 #8 끝: 1706ms
RSS 187MB
```

→ 리포트 하나: 1천만 건

© 한상훈 NestJS 백엔드 마스터

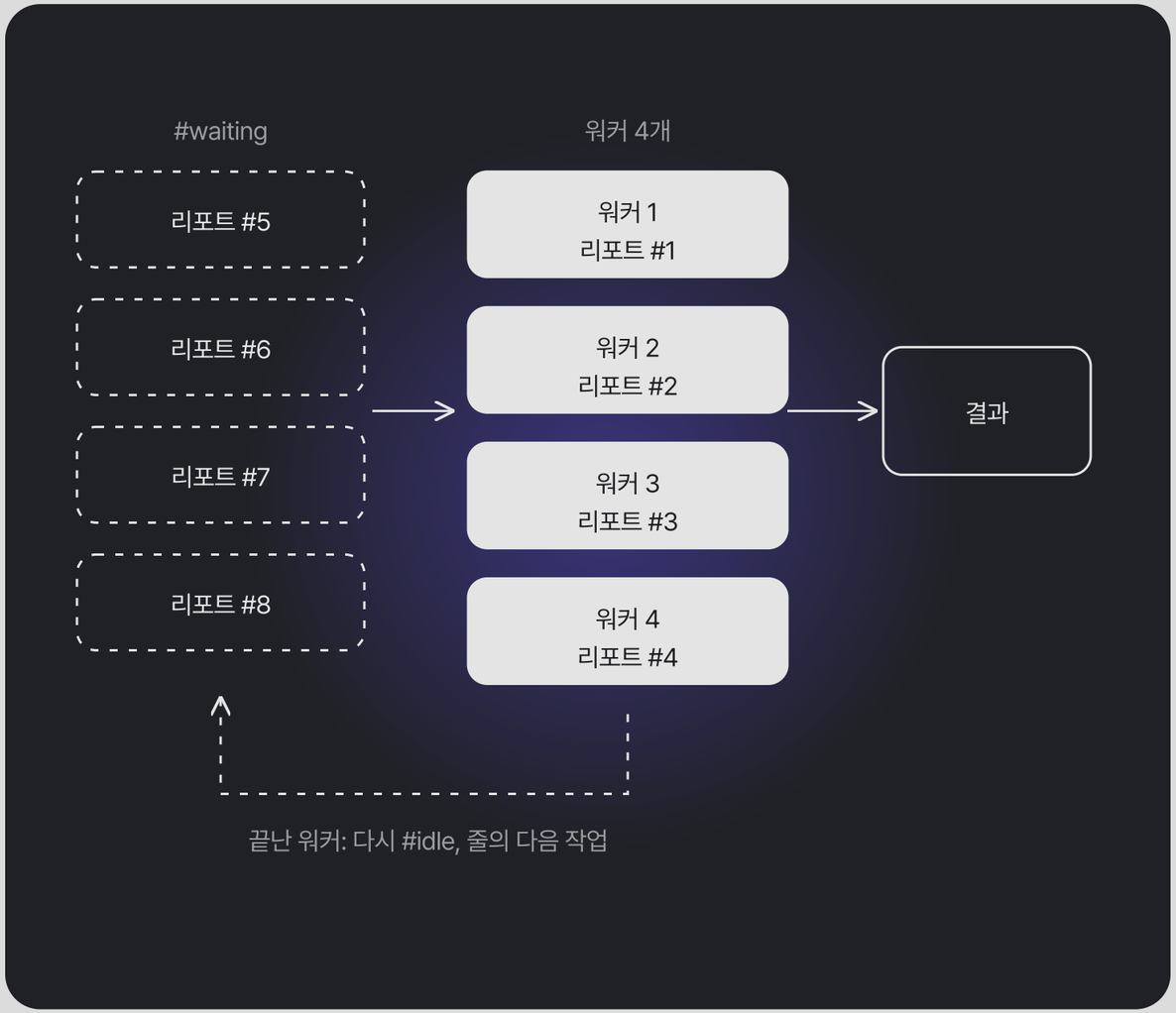
빈 워커를 기다리는 줄

→ 워커가 다 바쁘면: 작업은 `#waiting` 줄에 대기

→ 워커가 끝나면: 다시 `#idle`로, 줄의 다음 작업을 꺼냄

→ 워커 수 고정 → 메모리 고정

→ 줄 길이에도 한도: 넘으면 "잠시 뒤 다시 시도"로 거절



© 한상훈 NestJS 백엔드 마스터

풀 크기

→ `os.availableParallelism()`: 이 프로세스가 쓸 수 있는 CPU 수

→ CPU 수보다 많은 워커: 계산이 빨라지지 않고 서로 CPU를 나눠 씀

→ 메인 스레드 몫 남기기

→ 예: CPU 4개 서버 → 워커 2~3개로 시작, 루프 지연과 응답 시간 보며 조정

→ 서버 여러 대, 프로세스 여러 개면 그 수로 나눔

© 한상훈 NestJS 백엔드 마스터

작업 크기별 보낼 곳

작업	걸리는 시간	보낼 곳
작은 계산 (DTO 변환, 작은 JSON)	수 밀리초 이하	메인 스레드 그대로
리포트, 엑셀 생성, 큰 파싱	수백 밀리초~수 초	워커 풀
수십 초 이상, 재시작돼도 끝나야 하는 일	수십 초~	작업 큐와 별도 작업 프로세스

© 한상훈 NestJS 백엔드 마스터

비밀번호 해시와 이미지 처리

→ `crypto.pbkdf2`, `crypto.scrypt` 비동기 버전: 이미 libuv 스레드 풀에서 계산

→ bcrypt 패키지 비동기 함수: 스레드 풀

→ 이미지 처리 라이브러리: 자체 스레드를 쓰는 경우 많음

→ 먼저 확인: 쓰는 라이브러리가 이미 어디서 계산하는지

→ 모르면 측정: 메인 스레드 CPU 시간이 안 늘면 이미 다른 스레드

→ 응답 압축(zlib 비동기): 스레드 풀 사용, 파일·DNS와 같은 줄

© 한상훈 NestJS 백엔드 마스터

워커로 안 되는 것

→ 워커는 같은 프로세스 안: 프로세스가 죽으면 같이 종료

→ 배포로 서버가 재시작되면 계산 중인 리포트 소실

→ 요청 응답 안에서 끝나야 하는 일까지가 워커의 몫

작업 큐 흐름 예

작업 번호 즉시 응답 → 화면이 진행 상태 조회 → 완료 시 내려받기 링크·메일

© 한상훈 NestJS 백엔드 마스터

직원 두 종류

카페	Node	맡는 일
카운터 직원	이벤트 루프 (메인 스레드)	요청 받기, 기다리는 일 맡기기, 응답
뒷방 직원	워커 풀	오래 걸리는 우리 계산
가게 밖 공장	작업 큐와 작업 프로세스	수십 초 이상, 재시작돼도 끝나야 하는 일

© 한상훈 NestJS 백엔드 마스터

맡기는 곳

"기다리는 일은 이벤트 루프에 맡기고,
오래 계산하는 일은 워커에 맡깁니다."